

Punto 3.5 — Consultas de agregación: Count, Sum, Average, Min y Max

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.4. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Consultas de agregación: Count, Sum, Average, Min y Max** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.5
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ContarOrdenes`, `ContarOrdenesPorEstado`, `ExisteAlgunaOrden`, `TodasLasOrdenesTienenEstado`, `ObtenerPesoTotalDePlanchas`, `ObtenerPesoPromedioDePlanchas`, `ObtenerPesoMinimoDePlanchas`, `ObtenerPesoMaximoDePlanchas`, `ObtenerResumenPorCliente`, `ObtenerResumenPorEstado` y `ObtenerResumenMensual`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/AgregacionesUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class AgregacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public AgregacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== AGREGACIONES ===");
        var total = _unidad.Ordenes.ContarOrdenes();
        var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");
        var existe = _unidad.Ordenes.ExisteAlgunaOrden();
        var todasConEstado = _unidad.Ordenes.TodasLasOrdenesTienenEstado();
        var peso = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();
        var promedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();
        var minimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();
        var maximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();
        var porCliente = _unidad.Ordenes.ObtenerResumenPorCliente();
        var porEstado = _unidad.Ordenes.ObtenerResumenPorEstado();
        var mensual = _unidad.Ordenes.ObtenerResumenMensual();
        if (total != 5 || pendientes != 3 || !existe || !todasConEstado || peso != 1426.9m || promedio != 285.38m || minimo != 125.6m || maximo != 450.0m || porCliente.Count != 3 || porEstado.Count != 3 || mensual.Count != 5) throw new InvalidOperationException("Agregaciones inesperadas.");

        Console.WriteLine($"Órdenes: {total} | Pendientes: {pendientes} | Peso: {peso:N1} kg | Promedio: {promedio:N2} kg | Min: {minimo:N1} kg | Max: {maximo:N1} kg");
    }
}

```

Explicación línea a línea del caso de uso 3.5

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application que permiten consumir el repositorio sin referenciar EF Core desde el caso de uso.

Línea 3: `namespace AceriaData.Application.UseCases;` → Sitúa el caso de uso dentro de la capa Application.

Línea 5: `public sealed class AgregacionesUseCase` → Declara el caso de uso que valida las agregaciones del punto 3.5.

Línea 6: `{` → Abre el bloque de la clase.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Conserva la unidad de trabajo inyectada para acceder al repositorio.

Línea 8: `public AgregacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la dependencia mediante inyección de constructor.

Línea 10: `public void Ejecutar()` → Define la operación que ejecuta todas las agregaciones y sus aserciones E2E.

Línea 11: `{` → Abre el cuerpo del método Ejecutar.

Línea 12: `Console.WriteLine("=== AGREGACIONES ===");` → Identifica en consola el bloque de pruebas de agregación.

Línea 13: `var total = _unidad.Ordenes.ContarOrdenes();` → Ejecuta Count sobre las órdenes y conserva el total para validarlo.

Línea 14: `var pendientes = _unidad.Ordenes.ContarOrdenesPorEstado("Pendiente");` → Ejecuta Count con predicado para contar únicamente las órdenes pendientes.

Línea 15: `var existe = _unidad.Ordenes.ExisteAlgunaOrden();` → Ejecuta Any para comprobar existencia sin solicitar un recuento total.

Línea 16: `var todasConEstado = _unidad.Ordenes.TodasLasOrdenesTienenEstado();` → Ejecuta All para verificar que todas las órdenes cumplen la condición de tener estado no vacío.

Línea 17: `var peso = _unidad.Ordenes.ObtenerPesoTotalDePlanchas();` → Ejecuta Sum sobre los pesos y obtiene el total determinista del dataset.

Línea 18: `var promedio = _unidad.Ordenes.ObtenerPesoPromedioDePlanchas();` → Ejecuta Average sobre los pesos proyectados como anulables, con valor cero definido para una colección vacía.

Línea 19: `var minimo = _unidad.Ordenes.ObtenerPesoMinimoDePlanchas();` → Ejecuta Min y obtiene el peso mínimo de las planchas sembradas.

Línea 20: `var maximo = _unidad.Ordenes.ObtenerPesoMaximoDePlanchas();` → Ejecuta Max y obtiene el peso máximo de las planchas sembradas.

Línea 21: `var porCliente = _unidad.Ordenes.ObtenerResumenPorCliente();` → Agrupa por cliente y materializa los resúmenes agregados de cada grupo.

Línea 22: `var porEstado = _unidad.Ordenes.ObtenerResumenPorEstado();` → Agrupa por estado y materializa un resumen por cada estado presente.

Línea 23: `var mensual = _unidad.Ordenes.ObtenerResumenMensual();` → Agrupa por año y mes para validar GroupBy con claves temporales y agregados.

Línea 24: `if (total != 5 || pendientes != 3 || !existe || !todasConEstado || peso != 1426.9m || promedio != 285.38m || minimo != 125.6m || maximo != 450.0m || porCliente.Count != 3 || porEstado.Count != 3 || mensual.Count != 5) throw new InvalidOperationException("Agregaciones inesperadas.");` → Contrasta Count, Any, All, Sum, Average, Min, Max y los GroupBy contra el dataset determinista; cualquier desviación hace fallar el E2E.

Línea 25: `Console.WriteLine($"Órdenes: {total} | Pendientes: {pendientes} | Peso: {peso:N1} kg | Promedio: {promedio:N2} kg | Min: {minimo:N1} kg | Max: {maximo:N1} kg");` → Publica los valores agregados validados para facilitar la inspección manual de la ejecución.

Línea 26: `}` → Cierra el cuerpo del método Ejecutar.

Línea 27: `}` → Cierra la clase AgregacionesUseCase.

| Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<AgregacionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.5 OK");

```

Explicación línea a línea de Program.cs 3.5

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<AgregacionesUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<AgregacionesUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.5 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.5 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Average/Min/Max sobre una secuencia vacía requieren una estrategia explícita; el repositorio usa proyección nullable y coalescencia.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.5 - METODOS PEDAGOGICOS DE AGREGACION

```
// Se activan sólo en copias desechables para inspeccionar SQL y secuencias vacías.
decimal ObtenerPesoPromedioVacioSeguroReto();
decimal ObtenerPesoPromedioVacioSinEstrategiaReto();
string ObtenerSqlAgregadosReto();
string ObtenerSqlResumenMensualReto();
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.5 - METODOS PEDAGOGICOS DE AGREGACION

```

public decimal ObtenerPesoPromedioVacioSeguroReto() => _context.PlanchasAcero
    .Where(p => p.Id < 0)
    .Select(p => (decimal?)p.Peso)
    .Average() ?? 0m;

public decimal ObtenerPesoPromedioVacioSinEstrategiaReto() => _context.PlanchasAcero
    .Where(p => p.Id < 0)
    .Average(p => p.Peso);

public string ObtenerSqlAgregadosReto() => _context.PlanchasAcero
    .GroupBy(p => 1)
    .Select(g => new
    {
        Cantidad = g.Count(),
        Total = g.Sum(p => p.Peso),
        Promedio = g.Average(p => p.Peso),
        Minimo = g.Min(p => p.Peso),
        Maximo = g.Max(p => p.Peso)
    })
    .ToString();

public string ObtenerSqlResumenMensualReto() => _context.OrdenesFabricacion
    .AsNoTracking()
    .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
    .Select(g => new { Anio = g.Key.Year, Mes = g.Key.Month, TotalOrdenes = g.Count() })
    .OrderBy(x => x.Anio)
    .ThenBy(x => x.Mes)
    .ToString();

```

Archivo: src/AceriaData.Application/AgregacionesUseCase.cs

Marcador: ERROR CONTROLADO M03 3.5 - AGREGACION VACIA SIN ESTRATEGIA

```

// Contrasta el promedio no anulable sobre un conjunto vacío con la estrategia
// nullable + coalescencia usada por el checkpoint.
var promedioSeguroVacio = _unidad.Ordenes.ObtenerPesoPromedioVacioSeguroReto();
Exception? errorAgregacionVacia = null;

try
{
    _ = _unidad.Ordenes.ObtenerPesoPromedioVacioSinEstrategiaReto();
}
catch (Exception ex)
{
    errorAgregacionVacia = ex;
}

if (promedioSeguroVacio != 0m)
    throw new InvalidOperationException("Error controlado 3.5: la estrategia nullable debe
devolver 0 para el conjunto vacío.");

if (errorAgregacionVacia is null)
    throw new InvalidOperationException("Error controlado 3.5: la agregación no anulable sobre
el conjunto vacío no produjo el fallo esperado.");

Console.WriteLine($"Error controlado 3.5 OK | Seguro: {promedioSeguroVacio} | Sin estrategia:
{errorAgregacionVacia.GetType().Name}");

```

Comprobación esperada: la secuencia vacía obliga a definir explícitamente la semántica de Average/Min/Max.

Laboratorio adicional del punto 3.5

Diagnóstico técnico

Para agregaciones sobre conjuntos potencialmente vacíos, el repositorio proyecta a `decimal?` y aplica `?? 0m`. `Any()` expresa existencia y no debe sustituirse mecánicamente por `Count() > 0`.

Reto resuelto y verificación adicional

Calcula cardinalidades y peso total/promedio/mínimo/máximo y revisa que el SQL use agregados del servidor. Después agrupa por cliente y estado y contrasta los resultados con el dataset determinista.

Errores comunes revisados

Error	Causa	Solución
Average sobre colección vacía	Una sobrecarga no anulable puede fallar; una anulable puede devolver null	En AceriaData proyectar a <code>decimal?</code> y aplicar <code>?? 0m</code>
Min o Max sobre colección vacía	El resultado no anulable no tiene un valor válido que devolver	Proyectar a nullable y definir el valor del contrato, o comprobar existencia
GroupBy sin agregación	Se intenta tratar cualquier agrupación como <code>GROUP BY SQL</code>	Usar claves + agregados para la traducción relacional o revisar el <i>shape</i> real
HAVING mal expresado	El filtro agregado se aplica a las filas antes de agrupar	Aplicar <code>Where</code> sobre el agrupamiento para expresar el filtro de grupos
Count después de ToList	Se transfirieron filas solo para contarlas después en memoria	Si solo se necesita el total, llamar a <code>Count()</code> directamente sobre <code>IQueryable</code>
Agregación vacía	No se definió la semántica del resultado	Usar nullable + <code>??</code> cuando el contrato de AceriaData requiera un valor por defecto
Any con Count() > 0	Se usa Count en lugar de Any	Usar Any para mejor rendimiento

Analogía operativa

Las consultas de agregación en una acería son como los informes que el jefe de planta pide al archivo central. En lugar de revisar todas las carpetas una por una, el jefe pide un resumen: cuántas órdenes hay, cuánto pesan las planchas, cuál es la fecha más antigua. El archivo central calcula el resumen y lo entrega. Count es como contar las carpetas. Sum es como sumar los pesos. Average es como calcular el promedio. Min y Max son como encontrar el más ligero y el más pesado. GroupBy es como agrupar las carpetas por cliente o por estado. El filtro Having es como pedir solo los grupos que cumplen una condición. Las agregaciones sobre colecciones vacías son como pedir un promedio cuando no hay datos: hay que tener cuidado porque puede dar error. `DefaultIfEmpty` introduce un elemento por defecto en una secuencia vacía; el valor concreto y su traducción deben analizarse según el tipo y el proveedor. Así funcionan las agregaciones en EF Core: se calculan en el servidor, se devuelven resúmenes y se evita transferir todas las filas.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Calcula un resumen mensual y explica qué operaciones se ejecutan como agregados SQL.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.5 - METODOS PEDAGOGICOS DE AGREGACION

```
// Se activan sólo en copias desechables para inspeccionar SQL y secuencias vacías.
decimal ObtenerPesoPromedioVacioSeguroReto();
decimal ObtenerPesoPromedioVacioSinEstrategiaReto();
string ObtenerSqlAgregadosReto();
string ObtenerSqlResumenMensualReto();
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.5 - METODOS PEDAGOGICOS DE AGREGACION

```

public decimal ObtenerPesoPromedioVacioSeguroReto() => _context.PlanchasAcero
    .Where(p => p.Id < 0)
    .Select(p => (decimal?)p.Peso)
    .Average() ?? 0m;

public decimal ObtenerPesoPromedioVacioSinEstrategiaReto() => _context.PlanchasAcero
    .Where(p => p.Id < 0)
    .Average(p => p.Peso);

public string ObtenerSqlAgregadosReto() => _context.PlanchasAcero
    .GroupBy(p => 1)
    .Select(g => new
    {
        Cantidad = g.Count(),
        Total = g.Sum(p => p.Peso),
        Promedio = g.Average(p => p.Peso),
        Minimo = g.Min(p => p.Peso),
        Maximo = g.Max(p => p.Peso)
    })
    .ToString();

public string ObtenerSqlResumenMensualReto() => _context.OrdenesFabricacion
    .AsNoTracking()
    .GroupBy(o => new { o.FechaCreacion.Year, o.FechaCreacion.Month })
    .Select(g => new { Anio = g.Key.Year, Mes = g.Key.Month, TotalOrdenes = g.Count() })
    .OrderBy(x => x.Anio)
    .ThenBy(x => x.Mes)
    .ToString();

```

Archivo: src/AceriaData.Application/AgregacionesUseCase.cs

Marcador: RETO M03 3.5 - RESUMEN MENSUAL Y SQL DE AGREGADOS

```

// Valida el resumen mensual y contrasta las agregaciones del servidor.
var mensualReto = _unidad.Ordenes.ObtenerResumenMensual();
var norteReto = _unidad.Ordenes.ObtenerResumenPorCliente()
    .Single(x => x.Cliente == "Constructora del Norte");
var pendienteReto = _unidad.Ordenes.ObtenerResumenPorEstado()
    .Single(x => x.Estado == "Pendiente");

if (mensualReto.Count != 5 ||
    mensualReto.Any(x => x.Anio != 2024 || x.Mes < 1 || x.Mes > 5 || x.TotalOrdenes != 1))
    throw new InvalidOperationException("Reto 3.5: el resumen mensual no coincide con el
dataset.");

if (norteReto.TotalOrdenes != 3 || pendienteReto.TotalOrdenes != 3)
    throw new
InvalidOperationException("Reto 3.5: los resúmenes por cliente/estado no coinciden con el dataset.");

var sqlAgregadosReto = _unidad.Ordenes.ObtenerSqlAgregadosReto();
var sqlMensualReto = _unidad.Ordenes.ObtenerSqlResumenMensualReto();

Console.WriteLine($"Reto 3.5 OK | Meses: {mensualReto.Count} | Norte: {norteReto.TotalOrdenes} |
Pendiente: {pendienteReto.TotalOrdenes}");
Console.WriteLine("RETO_SQL_AGREGADOS_INICIO");
Console.WriteLine(sqlAgregadosReto);
Console.WriteLine("RETO_SQL_AGREGADOS_FIN");
Console.WriteLine("RETO_SQL_MENSUAL_INICIO");
Console.WriteLine(sqlMensualReto);
Console.WriteLine("RETO_SQL_MENSUAL_FIN");

```

Comprobación esperada: resumen mensual y SQL real de COUNT/SUM/AVG/MIN/MAX.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.5 OK, el siguiente estado parte exactamente de esta solución y añade **Aggregaciones con proyección**.